

SOFTWARE CONSTRUCTION AND DEVELOPMENT

Engr. Muhammad Umer Haroon

SOFTWARE CONSTRUCTION AND DEVELOPMENT

WEEK 5

RECALL LAST WEEKS

What Is a Tech Stack

- A tech stack is the collection of technologies used to build and run a software application. It usually has several layers.
- The front end is what the user sees, such as a web page or a mobile screen, and it is often built with technologies like React, Vue, or plain HTML and CSS.
- The back end is the logic and processing that happens on the server, and it is often built with languages like Python, Java, Node.js, or Go. The database is where data is stored, and common choices include PostgreSQL, MySQL, MongoDB, or SQLite.
- The infrastructure is where the application runs, such as a cloud provider, a physical server, or a container platform like Docker and Kubernetes.
- There are also supporting tools such as version control with Git, build tools, testing frameworks, and monitoring systems.

Tech Stack ...

- When people say "our tech stack," they usually mean the specific combination of these layers that a team has chosen.
- For example, a common stack for a small web project might be React on the front end, Node.js on the back end, PostgreSQL for the database, and deployment on a cloud platform like AWS or DigitalOcean.

Selection in Tech Stack ?

- Requirements ?
- Quality ?
- Architecture ?
- Bias ?

Why Tech Stack Selection Matters

- Choosing a tech stack is one of the earliest and most important decisions in a project. A good choice makes development faster, hiring easier, and maintenance cheaper.
- A bad choice can lock a team into a technology that is hard to hire for, expensive to run, or difficult to change later. This is sometimes called technical debt, meaning the future cost of a decision made today.
- The decision is hard because it is rarely purely technical. It also involves the skills of the team, the budget, the timeline, the expected scale, the long term maintenance plan, and even the preferences of the people who will maintain the system after the original developers leave.

Criteria for Choosing a Tech Stack

When evaluating a technology, a team should ask several questions.

- How mature is the technology, and has it been around long enough to be stable.
- How large and active is the community, because a large community means more tutorials, more libraries, and faster answers when problems arise.
- How easy is it to hire developers who already know this technology. How well does it fit the problem, since a technology that is perfect for one kind of application may be a poor fit for another.

- How well does it perform under the expected load. How secure is it, and how quickly are security issues fixed.
- How expensive is it to run, both in terms of licensing and in terms of cloud or server costs.
- How easy is it to integrate with the other parts of the stack.
- And finally, how likely is it to still be supported five years from now, because choosing a technology that is abandoned later is a painful experience.

Popular/common Stacks

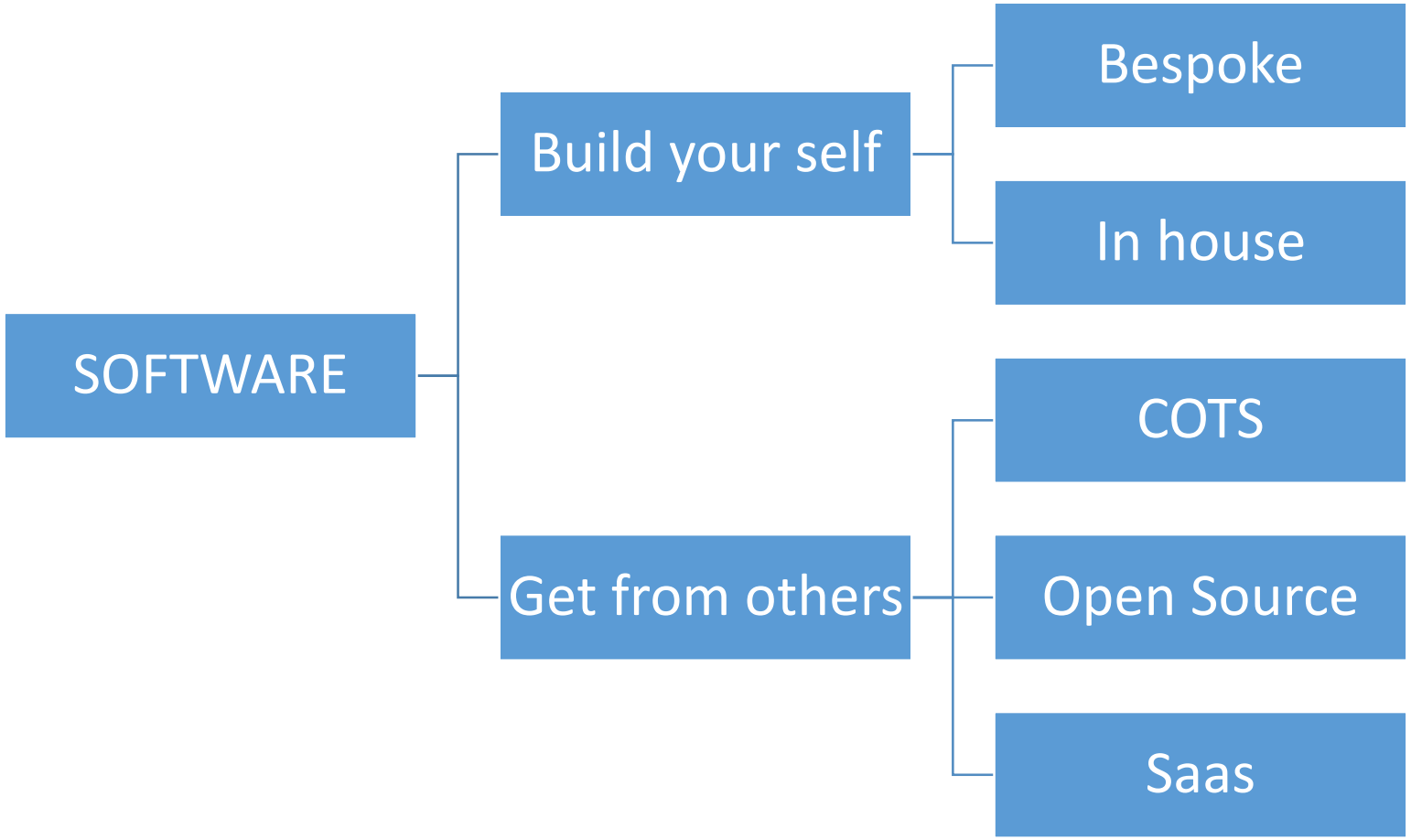
- The MERN stack, which stands for MongoDB, Express, React, and Node.js, is popular for JavaScript heavy web applications where the same language is used on both the front end and the back end.
- The LAMP stack, which stands for Linux, Apache, MySQL, and PHP, is a classic choice for content driven websites and has been used for decades.
- The Django stack, using Python with Django and PostgreSQL, is common for data heavy applications, internal tools, and projects where rapid development matters.
- The Spring stack, using Java with Spring Boot, is common in large enterprises where stability and long term support are critical.
- The .NET stack, using C# with ASP.NET, is common in organizations already invested in Microsoft technologies.

- There is no single best stack. There is only the best stack for a given team, project, and context.

Common Mistakes in Tech Stack Selection

- One common mistake is choosing a technology only because it is **trendy**.
- Another is choosing a technology because one developer on the team loves it, without considering whether the rest of the team can maintain it.
- Another is ignoring the cost of hiring, so the team picks a niche language and then cannot find developers.
- Another is overengineering, where a small project uses a complex stack meant for massive scale.
- And another is underestimating the cost of migration, meaning the team picks something now and later realizes it is very hard to move away from it.

Explore it yourself



Build vs Buy Decision

- A useful rule of thumb is to build the things that make your organization unique and give it a competitive advantage, and buy the things that are common across all organizations, such as email, payroll, or accounting.
- Another rule is to consider total cost of ownership, not just the purchase price.
- A cheap COTS product that requires heavy integration and customization may end up costing more than a custom solution.

Evaluating COTS Products

- When evaluating a COTS product, ask several questions. Does it meet the core functional requirements. How well does it integrate with existing systems.
- What is the total cost of ownership over five years, including licenses, support, training, and integration.
- How strong is the vendor, and how likely are they to still be around in five years.
- What is the exit strategy if you decide to leave, and how hard would it be to migrate your data out.
- How responsive is the vendor to security issues. And what do existing customers say about the product in reviews and forums.

CONTINUOUS IMPROVEMENT

In software construction and development, the act of writing code is only one part of the story.

Once a system is in the hands of users, it must be maintained, fixed, extended, and secured over time.

This ongoing process is called continuous improvement.

Read it

- Software is never truly finished. The moment a product is released, new information starts arriving. Users report bugs. Requirements shift as the business changes. New security threats emerge. Operating systems and libraries get updated. Competitors release features that raise user expectations. Hardware evolves. What worked perfectly six months ago may feel outdated today. A team that stops improving its software will watch it slowly decay in quality, security, and usefulness.

Release

- A release is a specific, packaged version of a software product that is made available to users. It is a snapshot of the codebase at a particular point in time, along with everything needed to install, run, and use it.
- A release is not just source code. It typically includes compiled binaries, configuration files, documentation, installation scripts, migration scripts for databases, and release notes that describe what has changed.

Types of Releases

- A major release introduces significant new functionality or makes changes that may break compatibility with previous versions. Users often need to plan for major releases because they may require configuration changes, retraining, or code updates.
- A minor release adds smaller features or improvements while remaining compatible with the previous version. Users can usually upgrade to a minor release with little risk.
- A patch release contains only bug fixes and security fixes, with no new features. Patch releases are meant to be safe and quick to apply.

- Some teams also distinguish between **alpha releases**, which are early and unstable; **beta releases**, which are more stable but still under testing; **release candidates**, which are believed to be ready but are given a final round of testing; and general availability releases, which are the official versions delivered to all users.

Release Cadences

A release cadence is the rhythm at which a team ships releases. Different teams choose different cadences depending on their product, their users, and their ability to test and deploy safely.

The right cadence depends on the product. Too frequent, and users feel overwhelmed and cannot keep up. Too infrequent, and users wait too long for fixes and features. A good team finds a rhythm that matches how its users actually work.

- Continuous deployment means changes are released automatically as soon as they pass tests, sometimes dozens of times per day. This is common in web based products where the team controls the entire environment.
- A fixed schedule means releases happen on a predictable timetable, such as every two weeks or every month. This is common in products where users need time to prepare for changes.
- A feature based cadence means releases happen when a set of features is ready, without a fixed date. This is common in smaller teams or in products with long development cycles.
- An on demand cadence means releases happen only when there is a compelling reason, such as a critical bug or a major new requirement. This is common in enterprise software and in products with infrequent updates.

Release Notes and Changelogs

- A release note is a document that accompanies a release and explains what is inside it. Good release notes are written for the user, not for the developer. They describe new features in plain language, list bug fixes, mention any breaking changes, and explain what users need to do to upgrade.
- A changelog is a running record of all releases, usually kept in the repository, that allows anyone to see the history of the product.
- Together, release notes and changelogs make it possible for users, support teams, and future developers to understand what happened and when.

Rollback and Canary Releases

- Even with careful testing, some releases cause problems in production. A rollback is the act of reverting to a previous version when a release causes issues. Good teams plan for rollbacks in advance, so that reverting is a quick and well rehearsed operation rather than a panicked scramble.
- A canary release is a technique where a new version is deployed to a small subset of users first, so that problems are caught before the release reaches everyone.
- If the canary group reports issues, the release can be stopped before it affects the full user base.

Versioning

Versioning

- Versioning is the practice of assigning a unique identifier to each release so that it can be referred to precisely. Without versioning, it would be impossible to know which copy of the software a user is running, which bugs have been fixed, or which features are available. Versioning is a communication tool.

Semantic Versioning

The most widely used versioning scheme is semantic versioning, often abbreviated as SemVer. In semantic versioning, a version number has three parts, written as MAJOR.MINOR.PATCH. For example, version 3.7.2 means major version 3, minor version 7, and patch version 2.

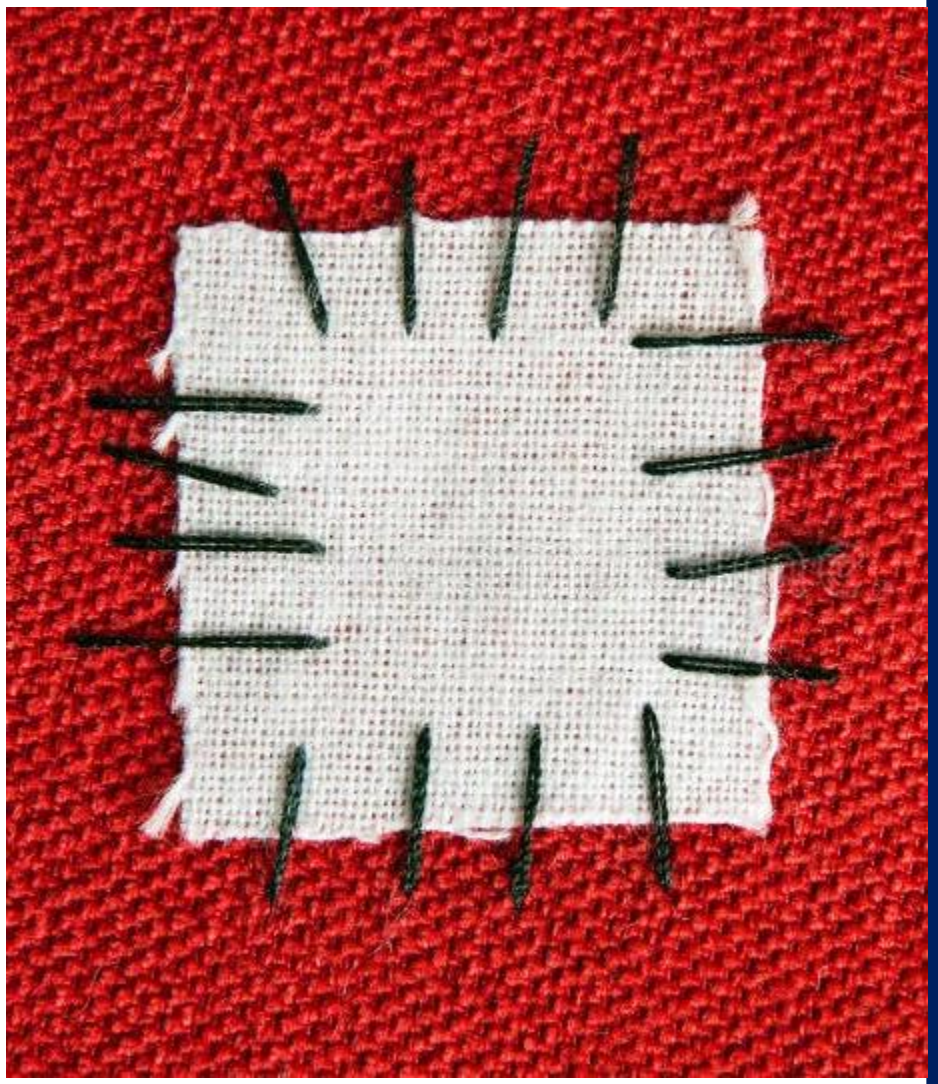
This scheme has a clear benefit. A user can look at a version number and immediately understand the risk of upgrading. Going from 3.7.2 to 3.7.5 is a patch update and is very low risk. Going from 3.7.2 to 3.8.0 adds features but should not break anything. Going from 3.7.2 to 4.0.0 signals that something may break and requires careful planning.

Alternative Versioning Schemes

- Not every project uses semantic versioning. Some projects use date based versioning, where the version is derived from the release date, such as 2024.09 or 20240915. This is common in projects with a regular release schedule, such as Ubuntu, which uses a year and month format. Some projects use calendar versioning combined with semantic hints.
- The important thing is not which scheme you choose, but that you choose one, apply it consistently, and document what it means. A version number that no one understands is worse than no version number at all.

Patch

- A patch is a small, targeted update that fixes a specific problem in a released version. Patches are most often used for security vulnerabilities and critical bugs, but they can also be used for smaller corrections. Unlike major or minor releases, patches are meant to be minimal. They change as little as possible so that the risk of introducing new problems is low.



Why patching ?

- Bugs exist in every system, and some of those bugs can be exploited by attackers. When a security vulnerability becomes public, attackers often move quickly to exploit it. If users do not apply the patch promptly, they remain exposed. This is why security patches are often released on an urgent schedule, sometimes within hours or days of a vulnerability being discovered.

The Tension Between Speed and Safety related to patching

- Patching involves a genuine tension. Security teams want patches applied as quickly as possible, because every hour of delay is an hour of exposure. Operations teams want patches tested thoroughly, because a bad patch can break production systems and cause outages that are worse than the original vulnerability.
- A good organization manages this tension through clear policies, automated testing, staged rollouts, and the ability to roll back quickly if something goes wrong. The goal is to manage intelligently.

The Cost of Not Patching

- Systems remain vulnerable to attacks, and a single unpatched vulnerability can lead to a data breach, financial loss, and reputational damage. Bugs accumulate, and users lose trust.
- The codebase drifts away from current best practices, making future changes harder. Eventually, the system becomes so outdated that it must be rewritten, which is far more expensive than steady maintenance would have been.
- This situation is sometimes called technical bankruptcy, and it is a real risk for any project that stops investing in its own health.

For queries

Office : B05, RSB

Counselling Hours :

Monday	11:00 am -12:00 pm
Tuesday	12:00 pm -1:00 pm
Thursday	10:15 am -11:15 am

mumer.haroon@imsciences.edu.pk

Muhammad Umer Haroon | LinkedIn